

Streams

Introduzione e Stream Pipeline

Cos'è uno Stream

Uno **Stream** (introdotto in **Java 8**, insieme a lambda e interfacce funzionali) è una **sequenza di dati che possiamo elaborare tramite operazioni**.

Il punto fondamentale da capire — e che la lezione ripete più volte — è questo: **uno Stream NON è una struttura dati**. Non è un array, non è una `List`, non è un `Set`. **Uno Stream non contiene dati**. È piuttosto un "tubo" attraverso cui i dati scorrono e vengono elaborati. La sorgente (l'array, la collezione, il file) contiene i dati; lo stream si limita a farli passare e trasformarli.

I vantaggi sono due:

- **Codice più conciso** (scrivi meno righe per elaborare i dati)
- **Efficienza**: grazie al meccanismo della *pipeline*, in certe situazioni l'elaborazione è molto più veloce.

La Stream Pipeline

Una **pipeline** è l'insieme delle operazioni che vengono eseguite su uno stream per produrre un risultato. È composta da **tre parti**:

1. **La sorgente (source)** — da dove provengono i dati: un array, una collezione, un file.
2. **Le operazioni intermedie (intermediate operations)** — trasformano lo stream in un altro stream. Possono essere **zero o più**. Ognuna prende uno stream e restituisce uno stream, quindi si possono concatenare.
3. **L'operazione terminale (terminal operation)** — è **obbligatoria** e serve a **far partire l'intero processo**, producendo il risultato finale.

 **Regola importante**: uno stream si può usare **una sola volta**. Dopo che l'operazione terminale è stata eseguita, lo stream è "esaurito". Se vuoi rielaborarlo, devi **ricrearlo** da capo.

Esempio con `filter`, `peek` e `count`

Il `filter` è un'operazione **intermedia**: riceve un **predicato** (una funzione che restituisce `true` o `false`) e lascia passare solo gli elementi per cui il predicato è vero.

Il `peek` serve a **osservare** il valore che scorre nello stream senza modificarlo — è un ottimo strumento di **debug**.

L'esempio della lezione: una lista di temperature. La pipeline fa:

```
List<Double> temps = Arrays.asList(98.4, 100.2, 87.9, 102.8);
```

```
long count = temps.stream()
```

```
.peek(t -> System.out.println(t))    // osservo il valore in ingresso
.filter(t -> t > 100)                // tengo solo > 100
.peek(t -> System.out.println(t))    // osservo cosa ha superato il filtro
.count();                           // operazione terminale
```

Cosa succede elemento per elemento (questo è il concetto chiave):

- 98.4 → primo peek lo stampa → il filtro lo scarta (non > 100). Il secondo peek NON lo vede.
- 100.2 → primo peek → passa il filtro → secondo peek lo stampa di nuovo (ecco perché vedi 100.2 due volte).
- 87.9 → primo peek → scartato.
- 102.8 → primo peek → passa → secondo peek.

Risultato finale: `count` = 2 (le due temperature > 100).

Stream Laziness (la "pigrizia" degli Stream)

Il principio di lazy evaluation

La **valutazione pigra (lazy evaluation)** significa: **ottieni ciò che ti serve solo quando ti serve.**

L'esempio della lezione: se devi mostrare 10.000 record a un utente, la valutazione pigra ne recupera ~50 mentre l'utente li guarda, e prepara i successivi 50 in background. La valutazione "impegnativa" (eager) caricherebbe invece tutti i 10.000 subito. Siccome magari l'utente ne guarda solo i primi, la lazy è molto più efficiente.

Come si applica agli Stream

Non succede NULLA finché non arriva l'operazione terminale. Le operazioni intermedie sono solo una "descrizione" di cosa fare; senza terminale, non si esegue niente.

Il grande vantaggio: siccome il JDK ha una **visione completa dell'intera pipeline**, può **evitare operazioni inutili**. Perché elaborare un dato se poi non serve? Ad esempio, se hai già trovato l'elemento che cercavi, o se hai un `limit` sul numero di elementi.

L'elaborazione è "verticale"

Questo è il punto più importante della lezione. Con `Stream.of(...)` creiamo uno stream:

```
Stream.of("Alex", "David", "April", "Edward")
    .filter(s -> true)           // lascia passare tutti
    .forEach(s -> System.out.println(s));
```

Gli elementi **non** vengono processati "orizzontalmente" (prima tutti nel filter, poi tutti nel forEach). Vengono processati **uno alla volta, verticalmente**: Alex attraversa filter → poi forEach; poi David attraversa filter → forEach; e così via.

Con map + anyMatch:

```
Stream.of("Alex", "David", "April", "Edward")
    .map(s -> s.toUpperCase())    // intermedia: trasforma
    .anyMatch(s -> s.startsWith("A")); // terminale in corto-circuito
```

Alex entra nel map → poi in anyMatch → parte per "A" → **match trovato, ci si ferma subito**. Se avessimo milioni di elementi, questo modo verticale fa **2 operazioni** invece di processare tutti nel map e poi tutti nel match. Ecco perché gli stream sono pigri ed efficienti.

Esempio completo con limit

```
Arrays.asList("April", "Ben", "Charlie", "David", "Christian").stream()
    .peek(...)                // osservo
    .filter(s -> s.startsWith("B") || s.startsWith("C")) // filtro 1
    .filter(s -> s.length() > 3) // filtro 2
    .limit(1)                  // intermedia stateful
    .forEach(...);            // terminale
```

Il **limit** è un'operazione intermedia **stateful** (mantiene uno stato: conta quanti elementi sono passati).

Traccia dell'esecuzione:

- **April**: passa il peek → filtro 1 (inizia con B o C? No) → **rimosso**.
- **Ben**: filtro 1 (inizia con B? Sì) → filtro 2 (lunghezza > 3? "Ben" ha 3, quindi No) → **rimosso**.
- **Charlie**: filtro 1 (C? Sì) → filtro 2 (> 3? Sì) → passa al forEach → viene stampato. **Ma limit(1)** ora dice: "ho raggiunto il mio limite, basta".
- Quindi **David e Christian non vengono nemmeno processati**: lo stream si ferma grazie al limit. Questa è la laziness in azione.

Creazione degli Stream (Creating Streams)

Ci sono diversi modi per creare uno stream:

Da un array

```
double[] nums = {1.1, 2.2, 3.3};
long count = Arrays.stream(nums).count(); // Arrays.stream()
```

Da una collezione (List, Set)

Il metodo `stream()` è un metodo **default** dell'interfaccia `Collection`:

```
List<String> list = Arrays.asList("a", "b", "c");
```

```
long count = list.stream().count();
```

Qui `count()` è l'operazione terminale che avvia tutto.

Da una Map

Attenzione: **una Map NON è una Collection**, quindi non ha direttamente `stream()`. Bisogna prima trasformarla in un set con `entrySet()` (che dà una vista-insieme della mappa), poi fare lo stream:

```
map.entrySet().stream().count();
```

Con Stream.of(...)

Metodo statico generico che accetta dei **var-args** e crea uno stream di quel tipo:

```
Stream.of(1, 2, 3).count(); // stream di Integer → 3
```

```
Stream.of("A", "B", "C", "D").count(); // stream di String → 4
```

Da un file

Il metodo `Files.lines(path)` riceve un `Path` e restituisce **uno stream di String**, dove ogni stringa è una riga del file:

```
try (Stream<String> lines = Files.lines(path)) {  
    lines.forEach(line -> { ... });  
}
```

L'esempio dei gatti: un file di testo con nome/colore (es. Fido/nero, Lily/bianco). Si legge riga per riga, si fa `line.split("/")` per ottenere `[nome, colore]`, si crea un oggetto `Cat` e lo si aggiunge a un `ArrayList`.

Due dettagli importanti dell'esempio:

- Esistono **due forEach diversi**: quello della **collezione** (`Collection.forEach`) e quello dello **stream** (`Stream.forEach`). Sono metodi diversi.
- La lista `cats` usata dentro una lambda **non può essere riassegnata** (deve essere `final` o *effectively final*): non posso farla puntare a un nuovo `ArrayList`. **Posso però** modificarne il contenuto (aggiungere gatti) — questo è un **effetto collaterale** consentito.

Stream infiniti

Stream.generate(Supplier) — riceve un `Supplier` (interfaccia funzionale, metodo `get()` che non prende nulla e restituisce un valore):

```
Stream.generate(() -> (int)(Math.random() * 10)) // numero casuale 0-9  
    .forEach(System.out::println); // va all'infinito finché non chiudi l'app
```

Rendere finito uno stream infinito con `limit`

```
Stream.iterate(2, n -> n + 2)
    .limit(10)           // corto-circuito stateful
    .forEach(System.out::println);
// Output: 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 (dieci numeri pari)
```

Operazioni Terminali (Terminal Operations)

Concetti generali

- Le operazioni terminali **avviano** il processo di streaming.
- Possono esistere **senza** operazioni intermedie, ma **non viceversa** (una pipeline senza terminale non fa nulla).

Il concetto di Riduzione (Reduction)

Una **riduzione** è un tipo speciale di operazione terminale in cui **tutti i contenuti dello stream vengono combinati in un unico valore** (una primitiva o un oggetto). Per essere una riduzione, l'operazione **deve elaborare tutti gli elementi**.

Ecco la classificazione delle operazioni terminali:

Operazione	Restituisce	È una riduzione?	Perché
<code>count()</code>	<code>long</code>	Sì	elabora tutto
<code>min()</code> / <code>max()</code>	<code>Optional</code>	Sì	elabora tutto
<code>average()</code>	<code>Optional</code>	Sì	elabora tutto
<code>findAny()</code> / <code>findFirst()</code>	<code>Optional</code>	No	possono fermarsi prima (corto-circuito)
<code>anyMatch</code> / <code>allMatch</code> / <code>noneMatch</code>	<code>boolean</code>	No	corto-circuito, non guardano tutto
<code>forEach()</code>	<code>void</code>	No	non restituisce nulla
<code>reduce()</code> / <code>collect()</code>	oggetto/ primitiva	Sì	elaborano e combinano tutto

`min` / `max` e `Optional`

`min` e `max` accettano un **Comparator** e restituiscono un **Optional**.

L'`Optional` è stato introdotto in Java per **sostituire `null`**. Se lo stream è vuoto, l'`Optional` è vuoto e non devi preoccuparti di `NullPointerException`.

```
Stream.of("dog", "pig", "horse")
    .min((s1, s2) -> s1.length() - s2.length()) // confronta per lunghezza
    .ifPresent(System.out::println); // stampa "pig" (parola più corta)
```

Se avessi filtrato prima (es. solo animali che iniziano per "S"), `min` restituirebbe un `Optional` vuoto e `ifPresent` non stamperebbe nulla — ecco a cosa serve l'`Optional`.

Stesso per `max` con dei numeri: se filtri via tutto (es. solo numeri > 20 quando non ce ne sono), l'`Optional` è vuoto.

findAny / findFirst

Restituiscono un `Optional`. **Non** sono riduzioni perché possono fermarsi prima di elaborare tutto.

Esempio: stream di "John" e "Paul". Cerco un risultato → trovo John → **Paul non viene elaborato**. Siccome uno stream si usa una volta sola, se voglio poi usare `findFirst` devo **ri-creare lo stream** da capo.

anyMatch / allMatch / noneMatch

Accettano un predicato e restituiscono un **boolean**:

```
List<String> names = Arrays.asList("Alan", "Brian", "Colin");
```

```
Predicate<String> startsA = s -> s.startsWith("A");
```

```
names.stream().anyMatch(startsA); // true → Alan
```

```
names.stream().allMatch(startsA); // false → Brian e Colin non iniziano per A
```

forEach

Restituisce `void`, quindi non è una riduzione. Se vuoi che accada qualcosa deve accadere **dentro il consumer** (effetto collaterale):

```
Stream.of("Cathy", "Pauline", "Zoe")
```

```
    .forEach(System.out::println);
```

🔴 **Dettaglio finale importante:** gli stream **non** possono essere la sorgente di un ciclo `for-each` potenziato, perché **gli stream non implementano l'interfaccia `Iterable`**. Un `for (Integer i : stream)` dà **errore di compilazione**. (Invece `Collection.forEach` sì, perché le collezioni implementano `Iterable`.)

Operazione Terminale — reduce()

`reduce( )` combina lo stream in un singolo oggetto. È **una riduzione** (elabora tutti gli elementi). È **sovraccaricato** (overloaded) in **tre versioni**.

Versione 1: `reduce(identity, accumulator)`

Firma: `T reduce(T identity, BinaryOperator<T> accumulator)`

- L'**identity** è il valore iniziale della riduzione e ciò che viene restituito se lo stream è vuoto. Quindi **c'è sempre un risultato** → non serve `Optional`.
- Il **BinaryOperator** (accumulatore) è un'interfaccia funzionale dove i due argomenti in ingresso e il tipo di ritorno sono tutti dello stesso tipo. Combina il risultato corrente con l'elemento successivo.

Esempio con stringhe — concateniamo "s", "e", "a", "n":

```
Stream.of("s","e","a","n")
    .reduce("", (a, b) -> a + b); // identity = stringa vuota
```

Accumulo: "" + "s" = "s" → "s" + "e" = "se" → "se" + "a" = "sea" → "sea" + "n" = "sean". Se lo stream fosse vuoto, restituirebbe "" (l'identity).

Esempio con numeri — moltiplichiamo 2, 3, 4:

```
Stream.of(2, 3, 4)
    .reduce(1, (a, b) -> a * b); // identity = 1
```

Accumulo: 1 * 2 = 2 → 2 * 3 = 6 → 6 * 4 = 24. (Nota: per la moltiplicazione l'identity giusta è 1, non 0.)

Versione 2: `reduce(accumulator)` — senza identity

Firma: `Optional<T> reduce(BinaryOperator<T> accumulator)`

Senza identity, restituisce un **Optional** perché potrebbero non esserci dati. Tre casi possibili:

1. Stream **vuoto** → `Optional` vuoto.
2. Stream con **un solo elemento** → restituisce quell'elemento (l'accumulatore non serve).
3. Stream con **più elementi** → applica l'accumulatore.

```
BinaryOperator<Integer> sum = (a, b) -> a + b;
```

```
Stream.<Integer>of().reduce(sum); // Optional vuoto → ifPresent non stampa
Stream.of(6).reduce(sum);       // restituisce 6
Stream.of(3, 4, 5).reduce(sum); // 3+4+5 = 12
```

Perché esistono due versioni (con e senza identity)?

Esempio illuminante: stream di 1, 1, 1 con `reduce((a, b) -> a)` (restituisce sempre il primo). Ottengo 1. Ma... quell'1 viene dall'accumulatore o dall'identity? Se avessi filtrato (solo numeri > 5) otterrei anch'io un risultato "vuoto/1". **A volte è utile distinguere tra "stream vuoto" e "risultato che coincide con l'identity"** — per questo esiste la versione con `Optional`.

Versione 3: `reduce(identity, accumulator, combiner)` — tipi diversi

Firma: `U reduce(U identity, BiFunction<U,T,U> accumulator, BinaryOperator<U> combiner)`

Serve quando i **tipi sono diversi** (l'elemento dello stream è di un tipo, il risultato di un altro). Permette di creare **risultati intermedi** da combinare alla fine — **utilissimo con gli stream paralleli**, dove lo stream viene spezzato e riassemblato da thread separati.

Esempio — contare i caratteri totali di alcune stringhe:

```
Stream.of("car", "bus", "train", "plane")
```

```
.reduce(0,                                // identity = 0 (Integer)
        (n, s) -> n + s.length(),         // accumulator: Integer + String → Integer
        (n1, n2) -> n1 + n2);           // combiner: unisce risultati parziali
```

- L'**accumulatore** mescola tipi diversi: prende un Integer e una String, restituisce un Integer.
- Il **combiner** unisce i risultati parziali: un thread calcola "car"+"bus" = 6, un altro "train"+"plane" = 14 → il combiner fa 6 + 14 = **20**.

Operazione Terminale — `collect()`

`collect()` è un tipo speciale di riduzione chiamata **riduzione mutabile (mutable reduction)**, perché usa lo **stesso oggetto mutabile** durante l'accumulazione (invece di crearne uno nuovo ogni volta). Questo la rende **più efficiente** delle riduzioni normali.

Oggetti mutabili comuni: `StringBuilder` e `ArrayList`.

È utilissima perché fa **uscire i dati dagli stream** e li mette in altre forme: **Map, List, Set**.

Versione manuale: `collect(supplier, accumulator, combiner)`

Si usa quando vuoi **controllo completo** (senza usare i Collector predefiniti):

- **Supplier**: non prende nulla, restituisce l'oggetto contenitore (es. `StringBuilder`).
- **Accumulator**: prende il contenitore + un elemento e lo aggiunge.
- **Combiner**: prende due contenitori e li unisce (utile in parallelo).

Esempio — concatenare "a", "d", "j", "u", "d", "i", "c", "a", "t", "e":

```
StringBuilder result = Stream.of("a", "d", "j", "u", "d", "i", "c", "a", "t", "e")
```

```
.collect(
    () -> new StringBuilder(),           // supplier
    (sb, s) -> sb.append(s),             // accumulator
    (sb1, sb2) -> sb1.append(sb2));     // combiner
```


// Un thread produce "adju", un altro "dicate" → combinati → "adjudicate"

collect() — Collectors.toMap()

Ora la versione **più importante** di collect: passare i **Collector predefiniti**, accessibili tramite metodi statici dell'interfaccia Collectors.

⚠ Questi metodi (Collectors.xxx) **non fanno nulla da soli**: vanno passati dentro collect().

Esempi rapidi:

- Collectors.joining(", ") → unisce le stringhe con un separatore: "cake, cookies, apple pie".
- Collectors.averagingInt(...) → prende una funzione che dà un int (es. la lunghezza) e ne calcola la **media**.

toMap() — due funzioni

Collectors.toMap() prende **due funzioni**:

1. La prima dice come creare la **chiave**.
2. La seconda dice come creare il **valore**.

Esempio — dal nome del dessert al numero di caratteri:

```
Stream.of("cake", "cookies", "apple pie")
    .collect(Collectors.toMap(
        s -> s,           // chiave = la stringa stessa
        s -> s.length()  // valore = la sua lunghezza
    ));
// {cake=4, cookies=7, apple pie=9}
```

Il problema delle chiavi duplicate → funzione di merge

Esempio inverso — dalla lunghezza al nome, con "cake" e "tart" che hanno **entrambi lunghezza 4**:

```
Stream.of("cake", "cookies", "tart")
    .collect(Collectors.toMap(
        s -> s.length(), // chiave = lunghezza ← COLLISIONE: 4 per cake e tart
        s -> s
    ));
// ✗ IllegalStateException: Duplicate key!
```

Java non sa cosa fare con due valori sulla stessa chiave. **Soluzione:** fornire una **funzione di merge** come terzo argomento:

```
Stream.of("cake", "cookies", "tart")
    .collect(Collectors.toMap(
        s -> s.length(),
        s -> s,
        (s1, s2) -> s1 + "," + s2 // merge: cosa fare coi valori in collisione
    ));
// {4=cake,tart, 7=cookies}
```

La funzione di merge dice **cosa fare con i valori** quando due chiavi collidono (qui li concatena con una virgola).

Scegliere il tipo di Map (es. TreeMap)

La mappa restituita è di solito una HashMap, **ma non è garantito**. Se vuoi una **TreeMap** (chiavi ordinate automaticamente), passi un **quarto argomento** (map supplier):

```
Stream.of("cake", "cookies", "apple", "TikTok")
    .collect(Collectors.toMap(
        s -> s.length(),
        s -> s,
        (s1, s2) -> s1 + s2,
        TreeMap::new // fornitore della mappa → TreeMap ordinata
    ));
```

collect() — Collectors.groupingBy()

groupingBy dice a collect di **raggruppare** tutti gli elementi in una **Map**:

- Prende **una funzione** che determina le **chiavi** della mappa.
- Ogni **valore** è una **List** di tutte le voci che corrispondono a quella chiave (List è il default).

Esempio — raggruppare nomi per lunghezza:

```
Stream.of("Joe", "Tom", "Tom", "Alan", "Peter")
    .collect(Collectors.groupingBy(s -> s.length()));
// {3=[Joe, Tom, Tom], 4=[Alan], 5=[Peter]}
```

Nota: "Tom" appare **due volte** perché il valore è una **List** (ammette duplicati).

Cambiare il tipo dei valori (List → Set) con un downstream collector

Se vuoi eliminare i duplicati usando un **Set** invece di una `List`, usi la versione overloaded con un **downstream collector**:

```
Stream.of("Joe", "Tom", "Tom", "Alan", "Peter")
    .collect(Collectors.groupingBy(
        s -> s.length(),
        Collectors.toSet()    // downstream: valori come Set
    ));
// {3=[Joe, Tom], 4=[Alan], 5=[Peter]} ← Tom una sola volta
```

Scegliere il tipo di Map (TreeMap) mantenendo List

Anche qui il tipo di mappa non è garantito. Per una `TreeMap` (chiavi ordinate) mantenendo i valori come `List`, si passano **entrambi** i parametri: il map supplier e il downstream collector:

```
Stream.of("Joe", "Tom", "Tom", "Alan", "Peter")
    .collect(Collectors.groupingBy(
        s -> s.length(),
        TreeMap::new,        // tipo mappa
        Collectors.toList()   // downstream
    ));
```

collect() — Collectors.partitioningBy()

`partitioningBy` è un caso speciale di `groupingBy` dove ci sono **solo due gruppi possibili**: `true` e `false`. Le chiavi della mappa sono i due **Boolean**. I valori sono di default una `List`.

Al posto di una funzione che dà la chiave, si passa un **predicato**.

Esempio 1 — nomi che iniziano per "T":

```
Stream.of("Thomas", "Theresa", "Mike", "Alan", "Peter")
    .collect(Collectors.partitioningBy(s -> s.startsWith("T")));
// {false=[Mike, Alan, Peter], true=[Thomas, Theresa]}
```

Esempio 2 — nomi con lunghezza > 4:

```
.collect(Collectors.partitioningBy(s -> s.length() > 4));
// true → [Thomas, Theresa, Peter], false → [Mike, Alan]
```

Esempio 3 — una partizione vuota. Se controlli nomi > 10 caratteri e nessuno lo è:

```
// true = [] (lista vuota), false = [Thomas, Theresa, Mike, Alan, Peter]
```

Nota che, a differenza di `groupBy`, `partitioningBy` include **sempre entrambe le chiavi** (`true` e `false`), anche se una corrisponde a una lista vuota.

Con downstream collector (Set) — anche `partitioningBy` accetta un downstream collector per cambiare List in Set:

```
Stream.of("Thomas", "Theresa", "Mike", "Alan", "Allen", "Peter")
    .collect(Collectors.partitioningBy(
        s -> s.length() > 4,
        Collectors.toSet()
    ));
```

```
// I duplicati (Allen inserito due volte) compaiono una sola volta nel Set
```

Operazioni Intermedie — `filter()`, `distinct()`, `limit()`

Premessa

A differenza di un'operazione terminale, **un'operazione intermedia produce sempre uno stream come risultato** (quindi si possono concatenare).

`filter()`

`filter` accetta un **predicato** e restituisce lo stream con **solo gli elementi che soddisfano il predicato**.

Esempio con contee irlandesi (Galway, Mayo, Roscommon): filtriamo per lunghezza del nome > 5.

```
Stream.of("Galway", "Mayo", "Roscommon")
```

```
    .filter(c -> c.length() > 5)
    .forEach(System.out::println);
```

```
// Passano: Galway (6) e Roscommon (9). Mayo (4) viene scartato.
```

Il `forEach` è l'operazione terminale che avvia tutto.

`distinct()`

`distinct()` restituisce uno stream con i **duplicati rimossi**, usando il metodo `equals()`. Quindi per le stringhe è **case-sensitive** (maiuscole e minuscole contano).

È un'operazione intermedia **stateful**: deve mantenere una memoria di ciò che ha già visto. Si comporta come un filtro:

- Se **non** ha mai visto l'elemento → lo lascia passare e lo **ricorda**.
- Se lo **ha già visto** → lo scarta.

Non prende argomenti, ma restituisce uno stream del tipo con cui stai lavorando (String, Integer, Person...).

Esempio con `peek` + `distinct` + `forEach`:

```
Stream.of("eagle", "eagle", "EAGLE")
    .peek(s -> System.out.println("peek: " + s))
    .distinct()
    .forEach(s -> System.out.println("output: " + s));
```

Traccia:

- "eagle" (minuscolo) → `peek` → `distinct` non l'ha mai visto → **lo ricorda** → output.
- "eagle" (minuscolo, seconda volta) → `peek` → `distinct` dice "già visto" → **scartato**, non compare in output.
- "EAGLE" (maiuscolo) → `peek` → `distinct` non l'ha mai visto (case-sensitive!) → passa → output.

Il punto: il secondo "eagle" minuscolo **non** appare due volte perché è stato filtrato, mentre "EAGLE" sì perché per `equals()` è diverso.

`limit()`

`limit` è un'operazione intermedia **stateful e in corto-circuito**: si **ferma** quando viene raggiunto il limite.

Esempio che mostra **sia il limit sia la lazy evaluation insieme**:

```
Stream.of(11, 22, 33, 44, 55, 66, 77, 88, 99)
    .peek(n -> System.out.println("A: " + n)) // punto A
    .filter(n -> n > 40)                      // solo > 40
    .peek(n -> System.out.println("B: " + n)) // punto B
    .limit(2)
    .forEach(n -> System.out.println("C: " + n)); // punto C
```

Traccia verticale, elemento per elemento:

- **11** → `peek A` → filtro (>40? No) → si ferma qui, non arriva a B.
- **22** → `peek A` → filtro No → si ferma.
- **33** → `peek A` → filtro No → si ferma.
- **44** → `peek A` → filtro (>40? Sì) → `peek B` → `limit` dice "questo è il 1° consentito" → punto C (output 44).
- **55** → `peek A` → filtro Sì → `peek B` → `limit` dice "questo è il 2° consentito" → punto C (output 55). **Il limite di 2 è raggiunto.**
- **66, 77, 88, 99** → **non vengono nemmeno prelevati dalla sorgente** (corto-circuito).

Questo è il cuore della lazy evaluation: una volta ottenuto il secondo elemento, si fa corto-circuito e si termina.

Operazioni Intermedie — `map()`, `flatMap()`, `sorted()`

`map()`

`map` crea una **mappatura** tra gli elementi dello stream e gli elementi dello stadio successivo. Serve a **trasformare i dati**. Accetta una **`Function<T, R>`** (T = tipo di input, R = tipo di ritorno), quindi restituisce uno stream di R.

```
Stream.of("book", "pen", "ruler")
    .map(s -> s.length())    // String → Integer
    .forEach(System.out::println);
// book → 4, pen → 3, ruler → 5. Output: 4, 3, 5
```

`map` ha trasformato uno **stream di String** in uno **stream di Integer**.

`flatMap()`

`flatMap` prende ogni elemento dello stream (che è **esso stesso un contenitore**, per esempio una lista di stringhe) e ne **"appiattisce"** il contenuto in **un unico stream**. Si passa da uno *stream di liste di stringhe* a uno *stream di stringhe*.

```
List<String> l1 = Arrays.asList("Shawn", "Desmond");
List<String> l2 = Arrays.asList("Mary", "Anne");

Stream.of(l1, l2)                // Stream<List<String>>
    .flatMap(list -> list.stream())    // → Stream<String>
    .forEach(System.out::println);
// Output: Shawn, Desmond, Mary, Anne
```

`sorted()`

`sorted()` restituisce lo stream con gli elementi **ordinati**. Come per l'ordinamento degli array, Java usa l'**ordine naturale** a meno che tu non fornisca un **Comparator**:

- Numeri → ordine ascendente.
- Stringhe → ordine alfanumerico.

È un'operazione intermedia **stateful**: deve vedere **tutti i dati** prima di poterli ordinare.

Con un **Comparator** (esempio con oggetti **Person**, John=23, Mary=25):

```
Stream.of(mary, john)
    .sorted((p1, p2) -> p1.getAge() - p2.getAge()) // lambda
// oppure equivalente con method reference:
```

```
.sorted(Comparator.comparingInt(Person::getAge))
.forEach(...);
```

Entrambi gli approcci (lambda o method reference `Person::getAge`) funzionano.

Esempio combinato (filter + sorted + limit)

```
Stream.of("Tim", "Jim", "Peter", "Wonderland", "Mary")
    .peek(...)           // peek 1: prima del filtro
    .filter(s -> s.length() == 3) // solo nomi di 3 caratteri
    .peek(...)           // peek 2: dopo il filtro
    .sorted()            // stateful
    .peek(...)           // peek 3: dopo l'ordinamento
    .limit(2)            // stateful, corto-circuito
    .forEach(...);
```

Traccia (concetto chiave: `sorted` deve raccogliere tutto prima di emettere):

- **Tim** → filtro (len 3? Sì) → entra nel `sorted` (che lo tiene in memoria).
- **Jim** → filtro Sì → `sorted` lo tiene.
- **Peter** → filtro (len 5) → **scartato**.
- **Wonderland** → filtro (len lungo) → **scartato**.
- **Mary** → filtro (len 4) → **scartato**.

A questo punto `sorted` ha raccolto [Tim, Jim], li ordina → [Jim, Tim]. Poi il `limit(2)`: Jim è il primo → esce; Tim è il secondo → esce (limite raggiunto, Java chiude la pipeline). Output ordinato: **Jim, Tim**.

Flussi Primitivi (Primitive Streams) — Creazione

Perché esistono

Oltre agli stream generici (`Stream<Integer>`, `Stream<Double>`, `Stream<Long>`), Java fornisce **classi di stream specializzate per le primitive**:

Classe	Lavora con le primitive
<code>IntStream</code>	<code>int</code> , <code>short</code> , <code>byte</code> , <code>char</code>
<code>DoubleStream</code>	<code>double</code> , <code>float</code>
<code>LongStream</code>	<code>long</code>

⚠ **Non confondere i nomi!** `IntStream` (primitiva `int`) è diverso da `Stream<Integer>` (oggetti `Integer`). L'indizio è nelle **parentesi angolari**: `Stream<Integer>` ha `<...>`, mentre gli stream primitivi **non le hanno mai**. `DoubleStream` = primitive, `Stream<Double>` = oggetti.

Creazione da array

```
int[] intArr = {1, 2, 3};
double[] dblArr = {1.1, 2.2};
long[] longArr = {10L, 20L};

IntStream is = Arrays.stream(intArr);    // → IntStream
DoubleStream ds = Arrays.stream(dblArr); // → DoubleStream
LongStream ls = Arrays.stream(longArr);  // → LongStream
```

Creazione con of

```
IntStream.of(1, 2, 3); // stream primitivo con 1, 2, 3
```

Metodi numerici automatici

Il grande vantaggio: gli stream primitivi hanno **metodi numerici già pronti** — `min`, `max`, `sum`, `average` — che con gli stream di oggetti dovresti costruire a mano.

Esempio con `reduce` (identity + `BinaryOperator`):

```
Stream.of(1, 2, 3)           // Stream<Integer>
    .reduce(0, (a, b) -> a + b); // identity 0 → somma = 6
```

Ma con lo stream primitivo puoi usare direttamente `sum()`:

```
Stream.of(1, 2, 3)
    .mapToInt(i -> i) // "spacchetta" Integer → int primitivo, dà un IntStream
    .sum();           // → 6
```

`mapToInt` prende una `ToIntFunction` (metodo `applyAsInt`), converte gli oggetti in primitive, e sull'`IntStream` risultante puoi chiamare `sum()`.

Flussi Primitivi — API (metodi principali)

sum vs gli altri

- **sum()** restituisce direttamente un valore primitivo (int, long o double) — **non** un Optional. Perché? Perché se lo stream è vuoto, **0 è un risultato valido** per una somma.
- **average()**, **max()**, **min()** restituiscono invece un **Optional**, perché lo stream potrebbe essere vuoto e non ci sarebbe un valore sensato.

Metodo	IntStream	LongStream	DoubleStream
sum()	int	long	double
max()	OptionalInt	OptionalLong	OptionalDouble
min()	OptionalInt	OptionalLong	OptionalDouble
average()	OptionalDouble	OptionalDouble	OptionalDouble

Nota: **average()** restituisce **sempre** un **OptionalDouble** (in tutti e tre i casi), per via delle possibili cifre decimali.

Esempi

```
// max su IntStream
```

```
IntStream.of(10, 20, 30)
    .max()           // OptionalInt
    .ifPresent(System.out::println); // stampa 30
```

```
// min con orElseThrow
```

```
DoubleStream.of(10, 20, 30)
    .min()           // OptionalDouble
    .orElseThrow();   // emette 10.0, o lancia eccezione se vuoto
```

```
// average con orElseGet (fornisce un fallback)
```

```
LongStream.of(10, 20, 30)
    .average()        // OptionalDouble
    .orElseGet(() -> Math.random()); // se presente 20.0, altrimenti numero casuale
```

I metodi **orElseThrow** e **orElseGet** sono modi alternativi di estrarre il valore dall'Optional: il primo lancia un'eccezione se vuoto, il secondo usa un **Supplier** come fallback.

Summary Statistics

Con ogni stream primitivo puoi ottenere **statistiche di riepilogo** in un colpo solo, tramite **summaryStatistics()**:

```
IntSummaryStatistics stats = IntStream.of(5, 10, 15, 20).summaryStatistics();
```

```
stats.getMin(); // 5 (su stream vuoto: il valore int più GRANDE possibile)
stats.getMax(); // 20 (su stream vuoto: il valore int più PICCOLO possibile)
stats.getAverage(); // 12.5 (su stream vuoto: 0.0)
stats.getCount(); // 4 (su stream vuoto: 0)
stats.getSum(); // 50 (su stream vuoto: 0)
```

Dettaglio utile: `getCount()` serve per capire se lo stream era vuoto (`count == 0`). E nota il comportamento "invertito" di `min/max` su stream vuoto (`min` restituisce il `max` int, `max` restituisce il `min` int) — è un valore sentinella.

Flussi Primitivi — Interfacce Funzionali

Con gli stream primitivi arrivano **versioni specializzate** delle interfacce funzionali che già conosci, per evitare autoboxing.

Supplier

- `Supplier<T>` → metodo `get()`, restituisce `T`.
- `DoubleSupplier` → `getAsDouble()` restituisce `double`.
- `IntSupplier` → `getAsInt()` restituisce `int`.
- `LongSupplier` → `getAsLong()` restituisce `long`.

Consumer

- `Consumer<T>` → `accept(T)`, ritorno `void`. `BiConsumer` → due argomenti.
- `DoubleConsumer` → `accept(double)`, `void`.
- `IntConsumer` → `accept(int)`, `void`.
- `LongConsumer` → `accept(long)`, `void`.

Predicate

- `Predicate<T>` → `test(T)` restituisce `boolean`.
- `DoublePredicate` → `test(double)` → `boolean`.
- `IntPredicate` → `test(int)` → `boolean`.
- `LongPredicate` → `test(long)` → `boolean`.

Function

- `Function<T,R>` → `apply(T)` restituisce `R`.
- `DoubleFunction<R>` → `apply(double)` restituisce `R`.
- `IntFunction<R>` → `apply(int)` restituisce `R`.
- `LongFunction<R>` → `apply(long)` restituisce `R`.

UnaryOperator / BinaryOperator

- `UnaryOperator<T>` → un argomento, ritorno **dello stesso tipo**.
- `DoubleUnaryOperator` → `applyAsDouble(double)` → `double`.

- `IntUnaryOperator` → `applyAsInt(int) → int`.
- `LongUnaryOperator` → `applyAsLong(long) → long`.
- Esistono anche le versioni **binarie** (`IntBinaryOperator`, ecc.): due argomenti dello stesso tipo, ritorno dello stesso tipo.

Il pattern da ricordare: prendi l'interfaccia base, aggiungi il prefisso del tipo primitivo (`Int/Long/Double`), e il metodo funzionale prende il suffisso `As<Tipo>` (`getAsInt`, `applyAsDouble`...).

Flussi Primitivi — Mappatura tra flussi

Questa lezione riguarda il **passaggio da un tipo di stream a un altro** tramite mappatura. L'API sembra complicata ma segue **uno schema**.

Il nome del metodo dipende dalla DESTINAZIONE

Vai verso...	Nome del metodo
Stesso tipo (stream → stream, int → int, ecc.)	<code>map</code>
Stream di oggetti	<code>mapToObj</code>
<code>DoubleStream</code>	<code>mapToDouble</code>
<code>IntStream</code>	<code>mapToInt</code>
<code>LongStream</code>	<code>mapToLong</code>

Il parametro (la funzione) dipende da SORGENTE e DESTINAZIONE

La regola generale: la funzione si chiama `<Sorgente>To<Destinazione>Function`.

- Da `IntStream` a `DoubleStream` → `IntToDoubleFunction` (metodo `applyAsDouble`).
- Da `DoubleStream` a `IntStream` → `DoubleToIntFunction` (metodo `applyAsInt`).
- Da `LongStream` a `DoubleStream` → `LongToDoubleFunction`.
- Ecc.

Casi particolari:

- Se **sorgente e destinazione sono lo stesso** primitivo → si usa `map` con un **`UnaryOperator`** (`IntUnaryOperator`, ecc.), non una funzione di conversione.
- Se vai da primitiva **a oggetti** → `mapToObj` con `IntFunction/DoubleFunction/LongFunction` (si **omette** il "To" nel nome della funzione).
- Se vai da oggetti a primitiva → `mapToInt/mapToDouble/mapToLong` con `ToIntFunction/ToDoubleFunction/ToLongFunction`.

Il "trucco" mnemonico: guarda **dove vuoi arrivare** per il nome del metodo, e guarda **da dove parti e dove arrivi** per il nome della funzione-parametro.

16. Codice — mappingObjectStreams()

Esempio pratico: partire da uno **stream di oggetti** (`Stream<String>`) e mapparli verso vari tipi, incluse le primitive.

Object → Object (map)

```
Stream.of("Ash", "Peach", "Sycamore")
    .map(s -> s.toUpperCase()) // Function<String,String>
    .forEach(System.out::println); // ASH, PEACH, SYCAMORE
```

Object → DoubleStream (mapToDouble)

```
Stream.of("Ash", "Peach", "Sycamore")
    .mapToDouble(s -> s.length()) // ToDoubleFunction: String → double
    .forEach(System.out::println); // 3.0, 5.0, 8.0
```

Qui la lunghezza (un `int`) viene automaticamente **up-cast** a `double`.

Object → IntStream (mapToInt)

```
Stream.of("tree")
    .mapToInt(s -> s.length()) // ToIntFunction: String → int
    ...                        // → 4
```

Object → LongStream (mapToLong)

```
Stream.of("tree")
    .mapToLong(s -> s.length()) // ToLongFunction: String → long (cast automatico)
    ...
```

Lo schema, ancora una volta: metodo = `map` + tipo destinazione; funzione = `To<Tipo>Function`.

Codice — mappingPrimitiveStreams()

Ora il contrario: partire da uno **stream primitivo** e mapparli verso altri tipi.

Primitiva → Object (mapToObj)

```
IntStream.of(1, 2, 3)
    .mapToObj(i -> "number " + i) // IntFunction<String>
    .forEach(System.out::println);
```

```
// "number 1", "number 2", "number 3"
```

Dopo il `forEach` lo **stream è chiuso**: per gli esempi successivi bisogna **ri-crearlo**.

Primitiva `int` → `DoubleStream` (`mapToDouble`)

```
IntStream.of(1, 2, 3)
    .mapToDouble(i -> i) // IntToDoubleFunction, cast automatico
    .forEach(System.out::println);
```

Primitiva `int` → `int` (map con `IntUnaryOperator`)

Quando sorgente e destinazione sono lo stesso primitivo, si usa `map`:

```
IntStream.of(1, 2, 3)
    .map(i -> i * 2) // IntUnaryOperator: int → int
    .forEach(...); // 2, 4, 6
```

Primitiva `int` → `LongStream` (`mapToLong`)

```
IntStream.of(1, 2, 3)
    .mapToLong(i -> i) // IntToLongFunction, cast automatico
    .forEach(...);
```

Optional

Cos'è

Pensa a un `Optional` come a una **scatola che può essere vuota o contenere un valore**. È stato introdotto per **sostituire `null`**: costringe il programmatore a **gestire esplicitamente** la possibilità che un valore non ci sia, invece di rischiare un `NullPointerException`.

Oltre a `Optional<T>` (per oggetti), esistono le versioni primitive: **`OptionalInt`**, **`OptionalDouble`**, **`OptionalLong`**.

Esempio: metodo che calcola una media

```
public static OptionalDouble calcAverage(int... nums) {
    if (nums.length == 0) return OptionalDouble.empty(); // scatola vuota
    int sum = 0;
    for (int n : nums) sum += n;
    return OptionalDouble.of((double) sum / nums.length);
}
```

Estrarre il valore — vari modi

get() diretto — pericoloso: se l'Optional è vuoto, lancia `NoSuchElementException`.

isPresent() + **get()** — l'approccio difensivo classico:

```
OptionalDouble opt = calcAverage(50, 60, 70);
```

```
if (opt.isPresent()) {  
    System.out.println(opt.getAsDouble()); // 60.0  
}
```

ifPresent(consumer) — stile più funzionale, non serve l'if:

```
opt.ifPresent(System.out::println); // esegue il consumer solo se c'è un valore
```

orElse(default) — fornisci un valore di fallback:

```
double val = opt.orElse(Double.NaN); // se vuoto → NaN, altrimenti il valore (60.0)
```

orElseGet(supplier) — fallback calcolato tramite un Supplier (usato quando lo stream è vuoto):

```
OptionalDouble empty = calcAverage(); // niente argomenti → vuoto
```

```
double val = empty.orElseGet(() -> Math.random()); // esegue il supplier
```

Pattern per gestire null con Optional

Un modo verboso:

```
Optional<String> opt;  
if (param == null) opt = Optional.empty();  
else opt = Optional.of(param);
```

L'API offre un metodo che fa **esattamente la stessa cosa** (come un operatore ternario incorporato):

Optional.ofNullable(param) — restituisce un Optional vuoto se il parametro è null, altrimenti un Optional con il valore.

```
Optional<String> opt = Optional.ofNullable(param);  
opt.ifPresent(System.out::println); // stampa solo se non era null
```

Optional primitivi in azione

```
IntStream.rangeClosed(1, 10) // 1..10 inclusi  
    .average() // OptionalDouble  
    .ifPresent(d -> System.out.println(d)); // DoubleConsumer → 5.5
```

```
OptionalDouble od = IntStream.rangeClosed(1, 10).average();  
double max = od.getAsDouble(); // 5.5 (spacchetta la primitiva double)
```

`getAsDouble()` è il metodo di `OptionalDouble` per estrarre la primitiva.

Flussi Paralleli (Parallel Streams)

Sequenziali vs Paralleli

Tutti gli stream visti finora sono **sequenziali**: elaborano un elemento **alla volta**.

Uno **stream parallelo** può elaborare gli elementi **simultaneamente** (allo stesso tempo). Java lo ottiene **spezzando lo stream in sotto-flussi (substreams)**, e le operazioni della pipeline girano sui sotto-flussi in parallelo, perché **ogni sotto-flusso può avere il proprio thread**.

Come creare uno stream parallelo — due modi

// Modo 1: `parallelStream()` dall'interfaccia `Collection`

```
List<Integer> list = List.of(10, 20, 30);
```

```
list.parallelStream()...
```

// Modo 2: `parallel()` dall'API `Stream`

```
list.stream().parallel()...
```

Esempio: somma

Versione sequenziale:

```
Stream.of(10, 20, 30, 40, 50, 60)
    .mapToInt(i -> i) // Stream<Integer> → IntStream
    .sum();           // 210
```

`mapToInt` prende una `ToIntFunction` (metodo `applyAsInt`); avresti potuto usare anche il method reference `Integer::intValue`.

Versione parallela — basta aggiungere `.parallel()`:

```
Stream.of(10, 20, 30, 40, 50, 60)
    .parallel()
    .mapToInt(i -> i)
    .sum();           // sempre 210
```

Cosa succede dietro le quinte

Sequenziale: $10+20=30$, $+30=60$, $+40=100$, $+50=150$, $+60=210$ (uno dopo l'altro).

Parallelo: lo stream viene spezzato, per esempio in due sotto-flussi:

- Thread 1: $[10, 20, 30] \rightarrow 60$

- Thread 2: [40, 50, 60] → 150
- Poi i due risultati parziali vengono **combinati**: $60 + 150 = 210$.

(Questo è esattamente il motivo per cui `reduce` e `collect` hanno una versione con **combiner**: serve a ricomporre i risultati parziali dei thread.)

⚠ **Attenzione all'ordine**

Con i flussi paralleli, **se l'ordine è importante, fai attenzione**: l'ordine del risultato dipende da **quale thread finisce prima**, non dall'ordine originale.

// Sequenziale:

```
Stream.of("A", "B", "C").forEach(System.out::print);    // ABC
```

// Parallelo:

```
Stream.of("A", "B", "C").parallel().forEach(System.out::print); // es. BCA
```

L'output parallelo può risultare, per esempio, **B, C, A** invece di A, B, C. Quindi, se l'ordine conta per te, occhio all'uso di `forEach` con stream paralleli (esistono alternative come `forEachOrdered` per preservare l'ordine).